

CIPHER

Cryptographic Intelligence for Post-quantum Hazard Evaluation and Readiness Assessment

Raspberry Pi Hardware Assembly & Deployment Manual



TARGET HARDWARE

Raspberry Pi 4 Model B — 4GB

NETWORK CONFIGURATION

1 × Built-in Wi-Fi (management)
1 × External USB Monitor Adapter

SOFTWARE BASELINE

CIPHER v0.1.0-dev — Phase 1 complete
(Windows)

MANUAL SCOPE

Physical assembly + Pi bring-up of the existing pipeline

HOW TO READ THIS MANUAL

This manual follows one rule above all others: **it never invents hardware, GPIO pins, network architecture, or software behaviour that does not already exist in the CIPHER repository.** Every page is labelled with one of the status markers below, taken directly from a repository audit performed before this manual was written (see the Repository Audit Summary page).

• CURRENTLY IMPLEMENTED

Verified, working code exists in the repository today and is covered by the test suite. Safe to rely on as-is.

HARDWARE INTEGRATION PENDING

The physical component (OLED, LEDs) is part of the approved bill of materials, but no CIPHER driver code exists yet to use it.

• SOFTWARE IMPLEMENTATION PENDING

The capability (live capture, Linux enforcement, systemd) is architecturally planned but not yet coded.

• GPIO ASSIGNMENT PENDING

No pin numbers are frozen or approved anywhere in the repository. Do not wire until a future revision assigns them.

• REFERENCE

A factual hardware specification (e.g. the Raspberry Pi's own pin numbering) shown for context only — not a CIPHER decision.

VISUAL CONVENTIONS



Numbered steps

Large circular numbers mark one physical action at a time, LEGO-manual style. Complete each step before moving to the next.

- ☐ Checklists confirm a state before you proceed.
- ☐ An empty box means "verify this yourself."

CAUTION

Amber panels flag a step that needs care but is not dangerous if done correctly.

STOP / DANGER

Red panels mark an action that can damage hardware or break the project's fixed constraints.

EXPECTED RESULT

Green panels describe what you should see if a step worked.

DOCUMENT INFORMATION

FIELD	VALUE
Manual version	1.1 (consistency-audited 1 October 2026)
Generated	28 September 2026
Source repository state	cipher-pqc, branch master, software v0.1.0-dev
Test baselines	1192 passing on the Windows development machine (current). 649/649 passing on the Pi at the earlier pre-reviewer-evaluation baseline; the 1192 baseline has not yet been pulled and re-verified on the Pi.
Primary sources consulted	README.md, docs/SDD.md, docs/DEMO_GUIDE.md, and every listed source module (capture/, enforcement/, config/, ml/, dashboard/, reports/)
Key assumption	No GPIO pins, OLED wiring, LED wiring, systemd unit, or Linux firewall policy is approved anywhere in the repository as of this writing — all are marked PENDING rather than guessed.
Fixed constraint carried throughout	Exactly one external USB Wi-Fi adapter. Built-in Wi-Fi is always the management interface.

ORIENTATION

04	Repository Audit Summary	4
05	System Overview	5
06	Network Architecture	6
07	Bill of Materials	7

COMPONENT IDENTIFICATION

08	Raspberry Pi 4 Board	8
09	GPIO Header, Pin 1 & Wi-Fi Roles	9
10	OLED, LEDs & Passives	10

BEFORE YOU POWER ON

11	Safety Before You Begin	11
-----------	-------------------------	----

OPERATING SYSTEM & NETWORK BRING-UP

12	MicroSD Card & OS Setup (Windows)	12
13	First Headless Boot & Verify Management Wi-Fi	13
14	Connect the Single USB Wi-Fi Adapter	14
15	Check Monitor-Mode Support	15

SOFTWARE BRING-UP

16	Clone CIPHER onto the Pi	16
17	Create a New Pi Virtual Environment	17
18	Verify ARM64 Software Compatibility	18
19	Software Validation Before GPIO	19
20	Isolation Forest ML Model on the Pi	20
21	Dashboard Access from the Windows Laptop	21

GPIO, OLED & LEDs (PENDING WORK)

22	GPIO Assignment — Pending Final Approval	22
23	OLED Hardware — Identification & Pin Map Pending	23
24	LED Hardware — Polarity, Resistors & Behaviour	24
25	Complete GPIO Assembly — Pending	25

SOFTWARE ARCHITECTURE REFERENCE

26	Live Multi-Device Capture — Intended Architecture	26
27	Reporting, Signing & Enforcement Logic	27
28	OLED/LED Software Tests & systemd Boot Concept	28

FINAL ASSEMBLY & OPERATION

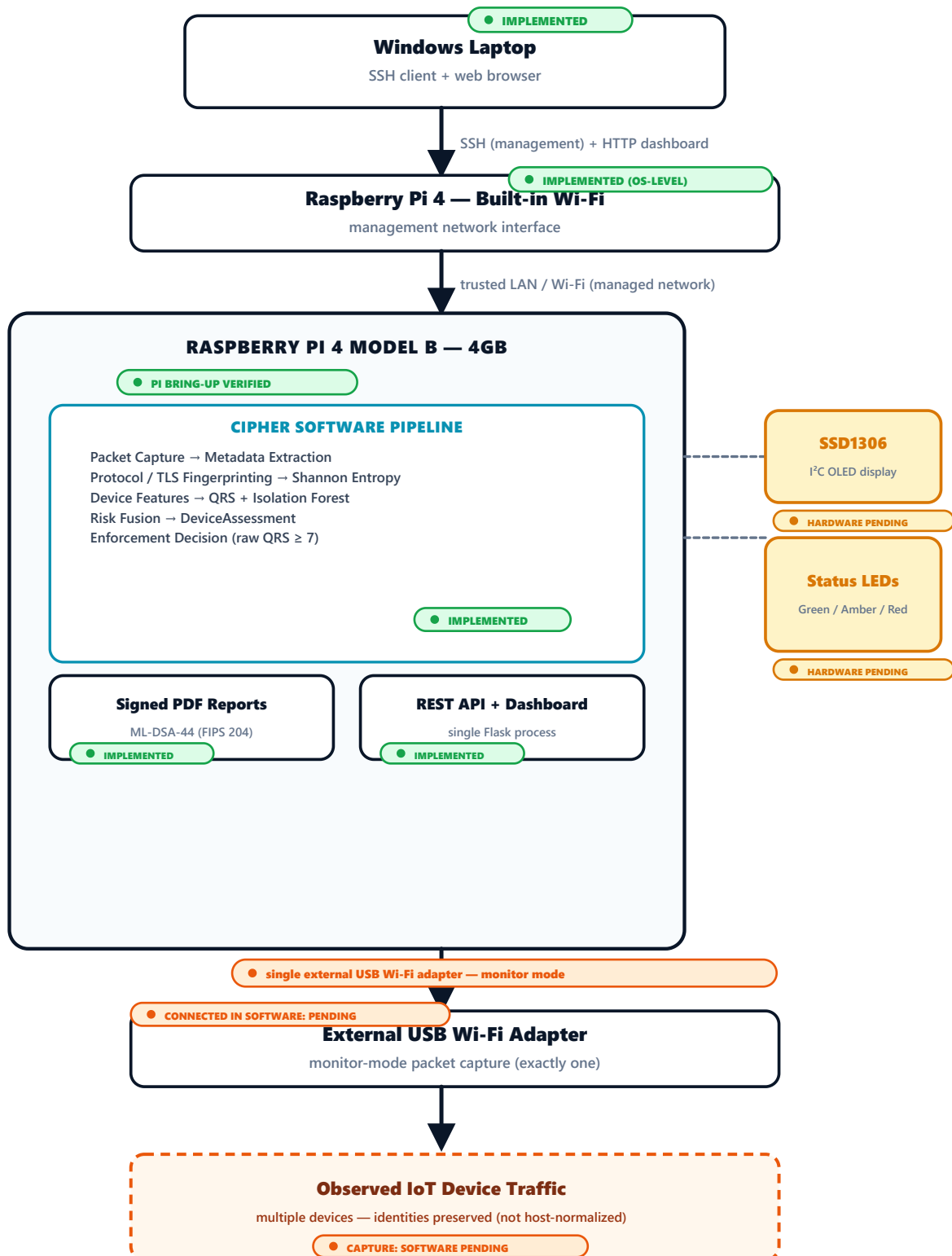
29	Final Physical Assembly — Part 1 of 2	29
30	Final Physical Assembly — Part 2 of 2	30
31	Power-On Checklist	31
32	Troubleshooting (I of II)	32
33	Troubleshooting (II of II) & Presentation Mode	33
34	Hardware & Software Status Table	34

Performed directly against the repository before any manual content was written. This table is the factual basis for every status pill used later in this document — nothing below is inferred or assumed.

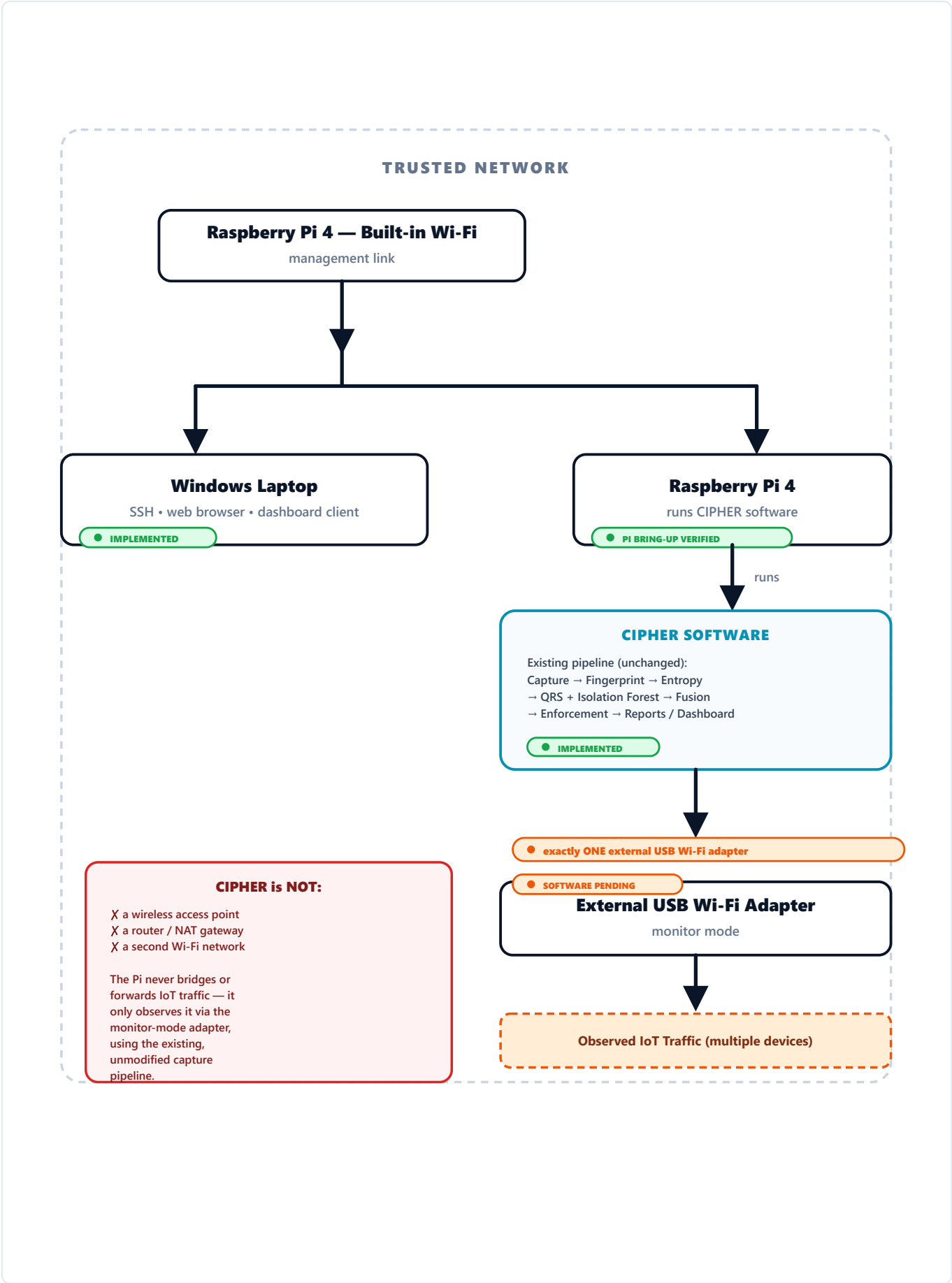
ITEM	IMPLEMENTED?	EXACT SOURCE	SAFE TO DOCUMENT AS FINAL?
Raspberry Pi OS-level bring-up	Yes (completed)	ARM64 Debian 13 on Pi 4 Model B 4GB; repository cloned, .venv built, pinned dependencies installed, imports succeeded, <code>preflight.py</code> passed, 649/649 tests passed at the then-current baseline	Yes — as bring-up only
OLED driver code	No	Not found anywhere (<code>grep -r ssd1306/oled</code> — zero matches)	No
GPIO LED driver code	No	Not found anywhere; no hardware/ package exists	No
GPIO pin assignments	No	No constant, config, or doc defines a pin number for any peripheral	No — marked PENDING
Windows host-level live capture	Yes (tested, Windows-only)	<code>capture/host_live_source.py</code> — a separate, temporary <code>run_live_demo.py</code> path; normalizes traffic around the local host; covered by <code>tests/capture/test_host_live_source.py</code>	Yes — document as its own, distinct capability
Raspberry Pi / multi-device live capture	No (scaffold only)	<code>capture/live_source.py</code> — raises <code>LiveCaptureNotImplementedError</code> immediately; this is the general, Pi-oriented scaffold, unaffected by the Windows path above	No
Monitor-mode hardware capability	Yes (verified outside CIPHER)	AR9271 on <code>ath9k_htc</code> (firmware 1.3) switched to monitor mode; 802.11 + Radiotap frames captured with <code>tcpdump</code> ; 100-packet PCAP written	Yes — as a hardware capability only
Monitor-mode handling inside CIPHER	No	Not referenced anywhere in <code>capture/</code> ; no Radiotap/802.11 parsing exists	No — ingestion not validated
Linux enforcement backend	No	<code>enforcement/backends.py</code> — only <code>NoOpIsolationBackend</code> exists; module docstring explicitly defers iptables	No — do not apply firewall rules
systemd service unit	No	No <code>.service</code> file anywhere in the repository	No
Dashboard network binding	Configurable, defaults to localhost	<code>config/constants.py</code> — <code>DEFAULT_FLASK_HOST = "127.0.0.1"</code> , overridable via <code>FLASK_HOST</code> env var	Yes — document the existing override
ML model loading	Yes	<code>ml/loading.py</code> , <code>ml/train.py</code> — optional, QRS-only mode if absent	Yes
Report generation	Yes	<code>reports/pdf_generator.py</code> — ML-DSA-44 signed, 3-page PDF	Yes
Core risk pipeline (QRS + fusion + enforcement decision)	Yes	<code>risk/</code> , <code>fusion/</code> , <code>enforcement/decision.py</code> — 1192 tests passing on Windows	Yes

WHAT THIS MEANS FOR THIS MANUAL

Pages describing the existing pipeline, reports, dashboard, and enforcement *decision logic* are written as **CURRENTLY IMPLEMENTED**. Every page touching physical Pi hardware (OLED, LEDs, GPIO, monitor-mode capture, Linux firewall rules, systemd) is written as **PENDING** and describes the intended architecture only — never invented specifics.



End-to-end view: management path (top), the existing CIPHER software core (middle), and the pending monitoring path (bottom). See the Legend page for what each status pill means.

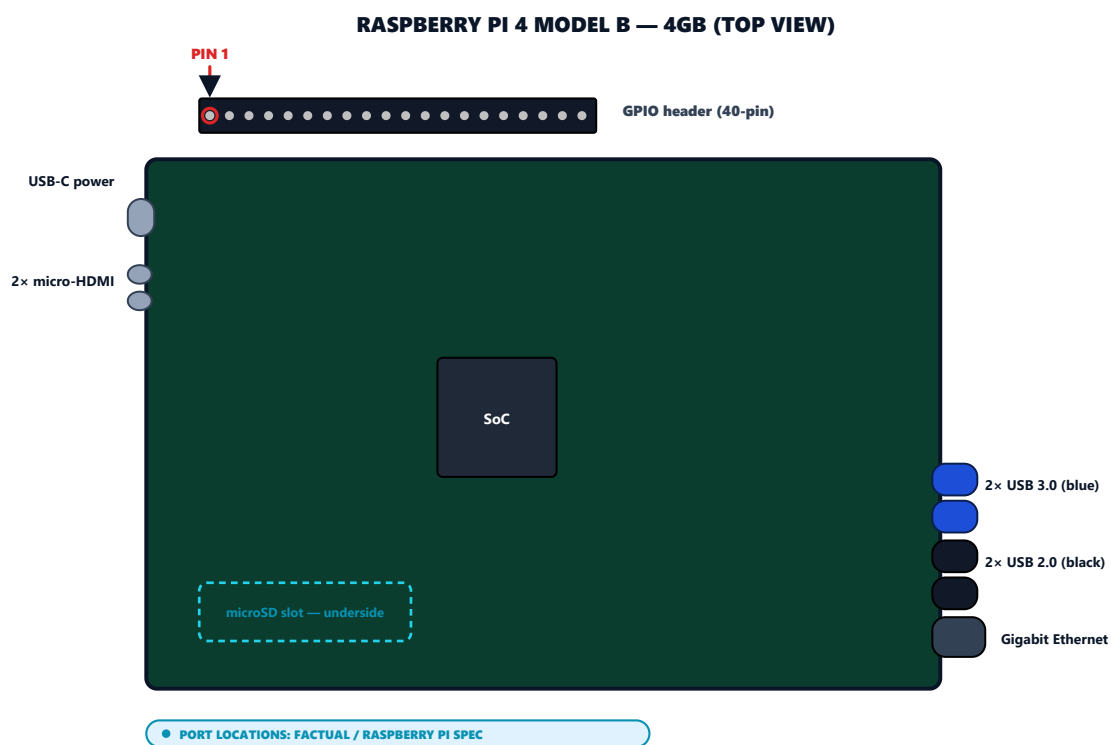


One trusted network. Built-in Wi-Fi carries management traffic; the single external USB adapter carries monitoring traffic. CIPHER never bridges, routes, or creates a second wireless network.

 Raspberry Pi 4 Model B 4GB RAM x1	 microSD card Raspberry Pi OS 64-bit x1	 Pi 4 USB-C power supply official / adequately rated x1
 External USB Wi-Fi adapter — Atheros AR9271 ath9k_htc fw 1.3; monitor mode verified — exactly one x1	 SSD1306 I²C OLED status display (pending) x1	 Green LED LOW risk indicator (pending) x1
 Amber/Yellow LED MEDIUM risk indicator (pending) x1	 Red LED HIGH risk indicator (pending) x1	 Current-limiting resistors one per LED, value per datasheet x3+
 Male-to-female jumper wires breadboard ↔ GPIO header as needed	 Breadboard only if it eases safe assembly x1 (if needed)	 Windows laptop SSH, dashboard, flashing microSD x1

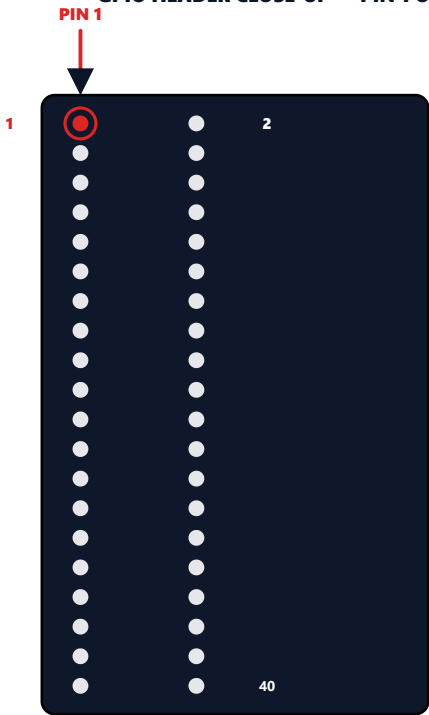
FIXED HARDWARE CONSTRAINT

This project can afford **exactly one** external USB Wi-Fi adapter. The Pi's **built-in Wi-Fi always handles management** (SSH, dashboard, package access) and is never repurposed for capture. Do not add, list, or budget for a second external adapter under any circumstance.



Port and connector locations are factual Raspberry Pi 4 Model B specifications, shown for identification only.

GPIO HEADER CLOSE-UP — PIN 1 ORIENTATION



Pin 1 sits at the corner of the 40-pin header nearest the USB-C power connector (a square solder pad on the underside marks Pin 1 on every Raspberry Pi board). Physical pin numbers count left-right, top-to-bottom: 1, 2 in the top row, down to 39, 40 at the far end.

● PORT / PIN-1 LOCATION: FACTUAL REFERENCE

This shows the physical header only — it does NOT show which pins CIPHER's OLED or LEDs use. See "GPIO Assignment — Pending Final Approval" before connecting anything here.

TWO WI-FI ROLES — EXACTLY ONE EXTERNAL ADAPTER



DO NOT ADD A SECOND EXTERNAL USB WI-FI ADAPTER

The project budget and design assume exactly one monitor-mode adapter. Built-in Wi-Fi always stays on the management network — never used for capture.



SSD1306 I²C OLED (0.96")

check silkscreen — label order varies by module

long leg = anode (+)



Green LED

long leg = anode (+)
flat side = cathode (-)

long leg = anode (+)



Amber LED

long leg = anode (+)
flat side = cathode (-)

long leg = anode (+)



Red LED

long leg = anode (+)
flat side = cathode (-)



Current-limiting resistor

(exact value per LED datasheet / supply voltage)



Male-to-female jumper wires

(male end → breadboard, female end → GPIO header)



Breadboard — only if needed for safe, easy assembly

● COMPONENT IDENTIFICATION ONLY — NOT A WIRING DIAGRAM

Identification only. None of these are wired yet — see "GPIO Assignment — Pending Final Approval."

General handling

- ☐ **Power the Pi off** (unplug the USB-C supply) before connecting or disconnecting any GPIO wiring.
- ☐ Use a suitable, adequately-rated official Raspberry Pi 4 USB-C power supply.
- ☐ Take basic static-discharge precautions — touch a grounded metal surface before handling the board.
- ☐ Do not experimentally move wires "to see what happens" while uncertain — stop and re-check the diagram instead.
- ☐ Double-check every connection against its labelled diagram before applying power.

Electrical specifics

- ☐ Raspberry Pi GPIO logic is **3.3 V** — never apply 5V logic directly to a GPIO pin.
- ☐ Avoid shorts between adjacent header pins, especially power and GND.
- ☐ Never connect an LED directly to a GPIO pin without a current-limiting resistor.
- ☐ LED polarity matters — the long leg (anode, +) and the flat-sided leg (cathode, -) are not interchangeable.
- ☐ Confirm you are using a GPIO pin, not a power pin (3V3/5V), before connecting a signal wire.

STOP IF UNSURE

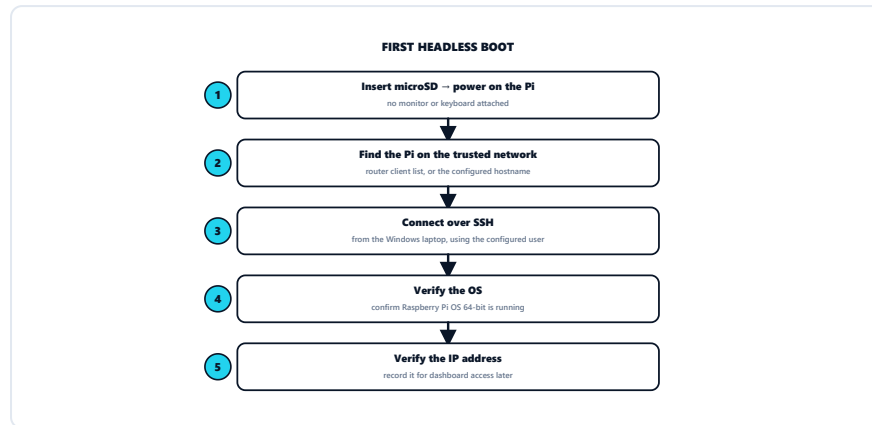
If a connection does not clearly match a labelled diagram in this manual, stop and re-check before applying power. GPIO wiring for the OLED and LEDs is currently **pending final approval** (see page 22) — do not improvise pin choices to "get it working" ahead of that approval.

WHY THIS MATTERS HERE SPECIFICALLY

This project has no committed GPIO wiring map yet. Every wiring mistake made while guessing pin numbers is a mistake this manual is specifically designed to prevent — follow the PENDING markers rather than working around them.

MICROSD / OS SETUP — FROM WINDOWS**NO HARDCODED CREDENTIALS**

Set your own hostname, username, and password in Raspberry Pi Imager's advanced options. This manual never supplies a default username/password — CIPHER's repository does not define one, and none should be invented here.



Find & connect

No monitor or keyboard is required. Find the Pi's address from your router's client list, or by its configured hostname, then connect from the Windows laptop:

```
ssh <your-username>@<hostname-  
or-ip>.local
```

Verify the built-in Wi-Fi is the management link

Before connecting the monitoring adapter, confirm the built-in Wi-Fi is up and identify its actual interface name — do not assume it will be wlan0.

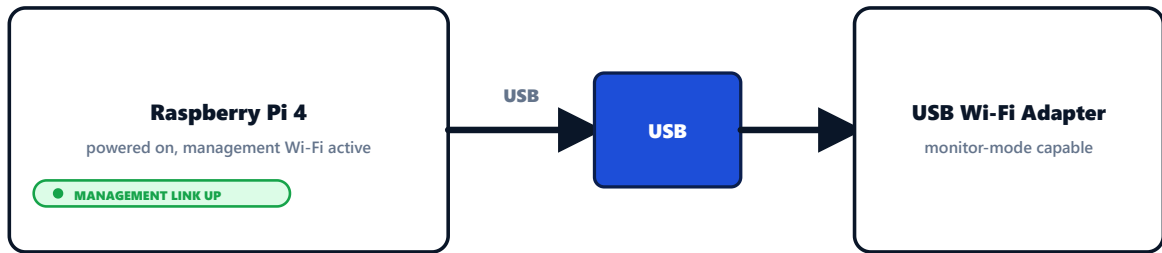
```
ip addr # confirm an IPv4 address on the built-in Wi-Fi interface  
ip link # confirm the interface is UP  
iw dev # confirm the interface is operating in managed mode
```

- ☐ Record the actual interface name shown (e.g. it may not be wlan0 on every board/OS image).
- ☐ Confirm it has a real IPv4 address on your trusted network.
- ☐ Confirm SSH from the Windows laptop still works over this same link.

EXPECTED RESULT

A working SSH session, a built-in Wi-Fi interface with an IP address on your trusted network, and a noted interface name you can refer back to later.

CONNECT THE SINGLE USB Wi-Fi ADAPTER



1. Connect the single USB Wi-Fi adapter to any free USB port.
2. Identify it from an SSH session:

```
lsusb  
ip link  
iw dev
```

Do not assume an interface name — record the actual name shown.

- ☐ Connect only the single approved external USB Wi-Fi adapter — never a second one.
- ☐ Identify it with `lsusb`, `ip link`, and `iw dev`; record its actual interface name.
- ☐ Confirm the built-in Wi-Fi management link is still connected and unaffected.
- ☐ Consider physically labelling the adapter (e.g. a small sticker) to avoid confusing it with the built-in interface later.

Before CIPHER uses the adapter, confirm the adapter, its driver, and monitor mode itself are all actually present — do not assume support for an unverified chipset.

```
iw list    # look for "monitor" under Supported interface modes
iw dev     # confirm the adapter's interface is listed
```

Checklist — completed on this deployment

- ☒ [x] USB adapter detected (appears in `lsusb`) — Qualcomm Atheros AR9271
- ☒ [x] Chipset identified — AR9271
- ☒ [x] Linux driver loaded and working — `ath9k_htc`, firmware 1.3
- ☒ [x] "monitor" mode listed as a supported interface type in `iw list`
- ☒ [x] Management Wi-Fi (built-in interface) still connected throughout this check

HARDWARE CAPABILITY VERIFIED

Monitor mode was enabled on the external adapter, and `tcpdump` captured IEEE 802.11 + Radiotap frames, writing a 100-packet PCAP file. This establishes that this adapter and driver can produce a valid monitor-mode capture on this Pi. It does **not** establish that CIPHER ingests such a capture — monitor-mode handling inside CIPHER remains **SOFTWARE IMPLEMENTATION PENDING**.

MONITOR MODE IS PASSIVE RECEPTION ONLY

Being in monitor mode does not mean the Pi automatically sees every device's usable traffic on an arbitrary Wi-Fi network: it receives frames on the channel it is tuned to, within radio range, and frames belonging to a protected network remain encrypted. CIPHER performs no packet injection, no deauthentication, and no credential capture. If you repeat this on different hardware and monitor mode is not listed, do not invent chipset-specific configuration — record the exact chipset and driver and treat monitor-mode capture as pending.

STATUS — TWO DIFFERENT MILESTONES, ONLY THE FIRST IS DONE

CIPHER's own capture pipeline does not yet consume a monitor-mode interface — `capture/live_source.py` is a documented scaffold that raises immediately, and no Radiotap/802.11 parsing exists anywhere in `capture/`. This page verifies the *adapter* is ready; wiring it into CIPHER's pipeline is separate, future work (see page 26).

Why a successful `tcpdump` capture is not CIPHER validation. `capture/offline_source.py` yields a packet only when it has an IP layer, a TCP or UDP layer, **and** a non-empty transport payload; anything else is silently skipped. Measured against the real, unmodified reader: a PCAP of encrypted (CCMP) 802.11 data frames yielded **0 of 5** packets, 802.11 beacons **0 of 5**, a mixed capture **0 of 4**, while *unencrypted* 802.11 data frames carrying IP/TCP via LLC/SNAP yielded **3 of 3**. So a monitor-mode PCAP is processable today only for its unencrypted-IP subset; a capture from a normal protected Wi-Fi network yields nothing, with no error.

1

Open an SSH session to the Pi

Using the management link confirmed on page 13.

2

Clone the repository

Use the project's actual repository URL:

```
git clone https://github.com/chandini-narayana/cipher-pqc.git
cd cipher-pqc
```

3

Authenticate securely if the repository requires it

Use an SSH key or a credential manager/personal access token entered interactively. Never paste a token, password, or private key into a script, commit, or this manual.

NEVER EXPOSE SECRETS

Do not embed tokens, passwords, or private keys in shell history, files committed to the repository, or screenshots. If a token is pasted into a terminal, clear scrollback/history afterwards.

DO NOT COPY AN EXISTING ENVIRONMENT

Do **not** copy `venv/` or `.venv-pi-validation/` from the Windows development machine. Both are Windows/x86-specific and gitignored — they will not work on ARM64 Linux.

1**Create a fresh environment from `requirements.txt`**

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

2**Confirm the environment is the one CIPHER will use**

Every subsequent command in this manual assumes this `.venv` is activated in the current shell.

WHY THIS MATTERS

`requirements.txt` is CIPHER's canonical, independently re-validated dependency baseline — but it was validated on Windows/x86 only. Raspberry Pi bring-up must build its own fresh environment from this same file, not by assuming Windows results transfer.

Windows validation does not prove Raspberry Pi compatibility. The pinned dependencies below must be verified installing cleanly on the actual Pi, from `requirements.txt` exactly as committed.

PACKAGE	PINNED VERSION	ARM64 INSTALL RESULT
flask	3.1.3	
scapy	2.7.0	
scikit-learn	1.8.0	
joblib	1.5.3	
dilithium-py	1.4.0	
cryptography	50.0.0	
reportlab	5.0.0	
python-dotenv	1.2.2	
pytest	9.1.1	
numpy	2.4.6	

IF A PACKAGE FAILS TO INSTALL

Record the actual failure (wheel unavailable, build-from-source error, etc.) before proposing any version change. Do not change a pinned version simply because installation is inconvenient — `requirements.txt` is CIPHER's canonical baseline for a reason (see `docs/SDD.md`'s Pre-Pi hardening addendum).

Failure notes (if any):

Run the existing, unmodified software baseline on the Pi before connecting or implementing any hardware-specific behaviour. Do not copy Windows performance numbers and relabel them as Pi results.

```
pytest -q          # full test suite
python preflight.py # environment/asset readiness check
python evaluate_demo.py # controlled 5-scenario evaluation harness
```

Results recorded on the Pi so far

Pi full test result: 649/649 passing — at the pre-reviewer-evaluation baseline

Pi preflight: passed, after training the Isolation Forest locally

Pi required imports: succeeded

Still outstanding on the Pi

Current 1192-test baseline: not yet pulled or re-verified on the Pi

Pi core scenarios (5/5):

Pi startup time:

Pi CPU utilisation:

Pi memory usage:

Pi temperature:

DO NOT REUSE WINDOWS FIGURES

The repository's measured performance figures are **Windows-only** measurements. Measure the Pi equivalents with the same harness (python evaluate_performance.py, and --stability-iterations for a sustained-run drift check), which runs unchanged on ARM64 and records platform metadata so the two stay distinguishable. **No Pi performance numbers have been collected yet** — do not quote any.

Isolation Forest anomaly detection is optional and secondary to QRS. If no trained model is present at `ml/artifacts/anomaly_detector.joblib`, CIPHER runs correctly in **QRS-only mode** — this is not an error.

If a trained model artifact is needed on the Pi

```
python -m ml.train # regenerates ml/artifacts/anomaly_detector.joblib
```

- ☐ This uses the existing, repository-approved synthetic training process (`ml/dataset.py`) — do not substitute a different data source.
- ☐ Do not silently substitute `data/datasets/CIPHER_Raw_Dataset_InputOnly_5000.xlsx` (the synthetic medical IoT workbook) as training data — the project's training process does not currently use it, and this manual does not change that decision.
- ☐ Prefer regenerating the deterministic model on the Pi using `python -m ml.train` so it is created under the Pi's installed Python/scikit-learn environment. If an existing artifact is reused instead, its compatibility must be explicitly verified.

DO NOT REDESIGN THE MODEL

This manual does not retrain, re-tune, or replace the existing Isolation Forest approach simply because the platform changed. If no model is present, CIPHER's QRS scoring is entirely unaffected — see page 27 for the fusion rule.

CIPHER serves its React dashboard and REST API from a single Flask process (`run_demo.py`). By default it binds to `127.0.0.1` — reachable only from the Pi itself.

1**Bind to the LAN, using the existing configuration surface**

This uses CIPHER's own settings mechanism (`config/settings.py`) — no code change.

```
export FLASK_HOST=0.0.0.0
python run_demo.py
```

2**Open the dashboard from the Windows laptop's browser**

```
http://<PI-IP>:5000
```

Using the Pi's built-in Wi-Fi management-link address recorded on page 13.

ONLY BIND AS WIDELY AS NEEDED

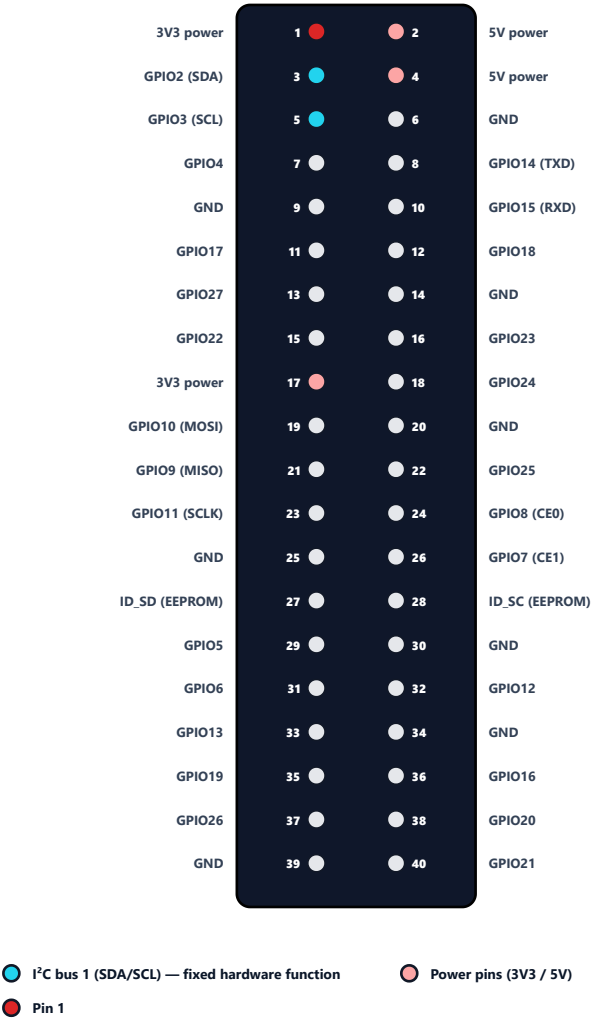
`FLASK_HOST=0.0.0.0` exposes the dashboard to the whole trusted LAN, not just your laptop. Do not expose it more broadly (e.g. port-forwarded to the public internet) — that is outside this project's scope.

NO GPIO PIN IS APPROVED YET

Nothing in the CIPHER repository assigns a physical or BCM pin number to the OLED (VCC/GND/SDA/SCL) or to any LED. This page intentionally does not present a wiring map as final.

RASPBERRY PI 4 — OFFICIAL 40-PIN HEADER (REFERENCE)

Factual hardware pinout — for context only. NOT a CIPHER wiring map.



- ☐ CIPHER should use the Raspberry Pi's standard I²C1 interface. Its default SDA/SCL connections are BCM GPIO2 / physical pin 3 and BCM GPIO3 / physical pin 5.
- ☐ Which GND and 3V3/5V pins power the OLED, and which GPIO pins drive the three LEDs, remain open decisions.
- ☐ Do not wire the OLED or LEDs until a future manual revision freezes these choices.

The approved bill of materials specifies an **SSD1306 I²C OLED**. Its four connections are typically labelled VCC, GND, SDA, SCL — but always verify the exact markings printed on your specific module before assuming label order.

CHECK THE MODULE ITSELF

Label order and silkscreen wording vary between SSD1306 breakout boards. Read your physical module before wiring anything, and never assume undocumented voltage tolerance — confirm the module is 3.3V-safe (most are, but verify).

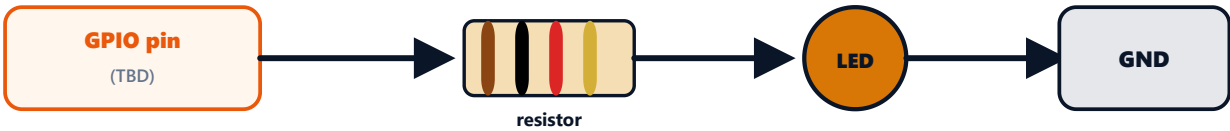
CIPHER OLED WIRING: PIN MAP PENDING

No final GND/power/SDA/SCL pin choice is approved in the repository yet. Do not invent final connections — wait for the GPIO Assignment page (22) to be updated in a future revision.

What the eventual wiring will require

- ☐ A ground connection (any Pi GND pin).
- ☐ A power connection matching the module's rated voltage (commonly 3.3V on the Pi header).
- ☐ CIPHER should use the Raspberry Pi's standard I²C1 interface. Its default SDA/SCL connections are BCM GPIO2 / physical pin 3 and BCM GPIO3 / physical pin 5.

LED WIRING CONCEPT — GPIO PIN TO BE ASSIGNED



GPIO → resistor → LED (anode) → LED (cathode) → GND

- Repeat once per LED colour (green / amber / red) once GPIO pins are approved.
- Never connect an LED directly to a GPIO pin without a resistor.
- Confirm LED polarity (long leg / flat side) before connecting.

Exact GPIO pin numbers are NOT assigned yet — see “GPIO Assignment — Pending”

Desired hardware behaviour (software not yet implemented)

CIPHER RISK CATEGORY	DESIRED LED COLOUR	STATUS
LOW	Green	• SOFTWARE PENDING
MEDIUM	Amber	• SOFTWARE PENDING
HIGH	Red	• SOFTWARE PENDING

DESIRED BEHAVIOUR, NOT CURRENT BEHAVIOUR

No GPIO control code exists yet anywhere in the repository. LEDs do not currently respond to any assessment. This table records the intended mapping for a future software phase — it is not a claim about today's software.

THIS PAGE IS INTENTIONALLY INCOMPLETE

A single, final illustration showing the Pi, OLED, all three LEDs, resistors, and jumper wires together — with a complete wiring table (Component / Signal / BCM / Physical pin / Destination) — can only be produced once the GPIO assignment on page 22 is genuinely frozen and approved.

What will appear here once GPIO pins are approved

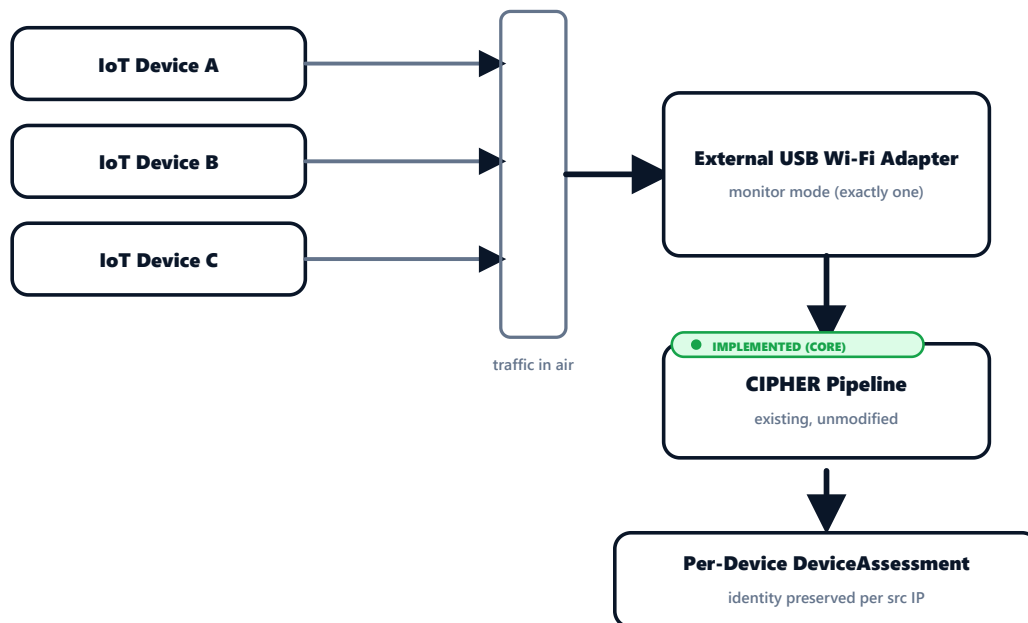
COMPONENT	SIGNAL	BCM	PHYSICAL PIN	DESTINATION
SSD1306 OLED	SDA	GPIO2	3	
SSD1306 OLED	SCL	GPIO3	5	
SSD1306 OLED	VCC			
SSD1306 OLED	GND			
Green LED (LOW)	anode via resistor			
Amber LED (MEDIUM)	anode via resistor			
Red LED (HIGH)	anode via resistor			

NEVER LEAVE AN AMBIGUOUS CONNECTION

Every blank cell above must be filled from an actually-approved decision before any wire is connected — never from a guess made to finish this table.

INTENDED MULTI-DEVICE CAPTURE ARCHITECTURE

● SOFTWARE IMPLEMENTATION PENDING



Must NOT reuse Windows host-live normalization — real device identities are preserved here

DO NOT REUSE THE WINDOWS HOST-LIVE APPROACH

The temporary Windows live-capture mode (`capture/host_live_source.py`) treats the laptop itself as the one monitored device and normalizes every packet around its own IP. That behaviour is specific to host-level capture and must **not** be copied into the Pi's multi-device monitoring architecture, which must preserve each observed device's real identity by source IP.

EXISTING RISK, REPORTING & ENFORCEMENT FLOW



QRS SCORE	CATEGORY	ISOLATION ELIGIBLE?
0-2	LOW	No
3-6	MEDIUM	No
7-10	HIGH	Yes (raw QRS ≥ 7)

OLED / LED software tests — IMPLEMENTATION PENDING

No OLED or LED driver code exists yet, so no repository-supported test command exists either. The checklist below records what will need verifying once that code exists — it does not name any command that exists today.

- ☐ [] OLED initializes and displays expected content over I²C — **PENDING**
- ☐ [] Green LED lights for a LOW-category assessment — **PENDING**
- ☐ [] Amber LED lights for a MEDIUM-category assessment — **PENDING**
- ☐ [] Red LED lights for a HIGH-category assessment — **PENDING**

systemd boot concept — IMPLEMENTATION PENDING

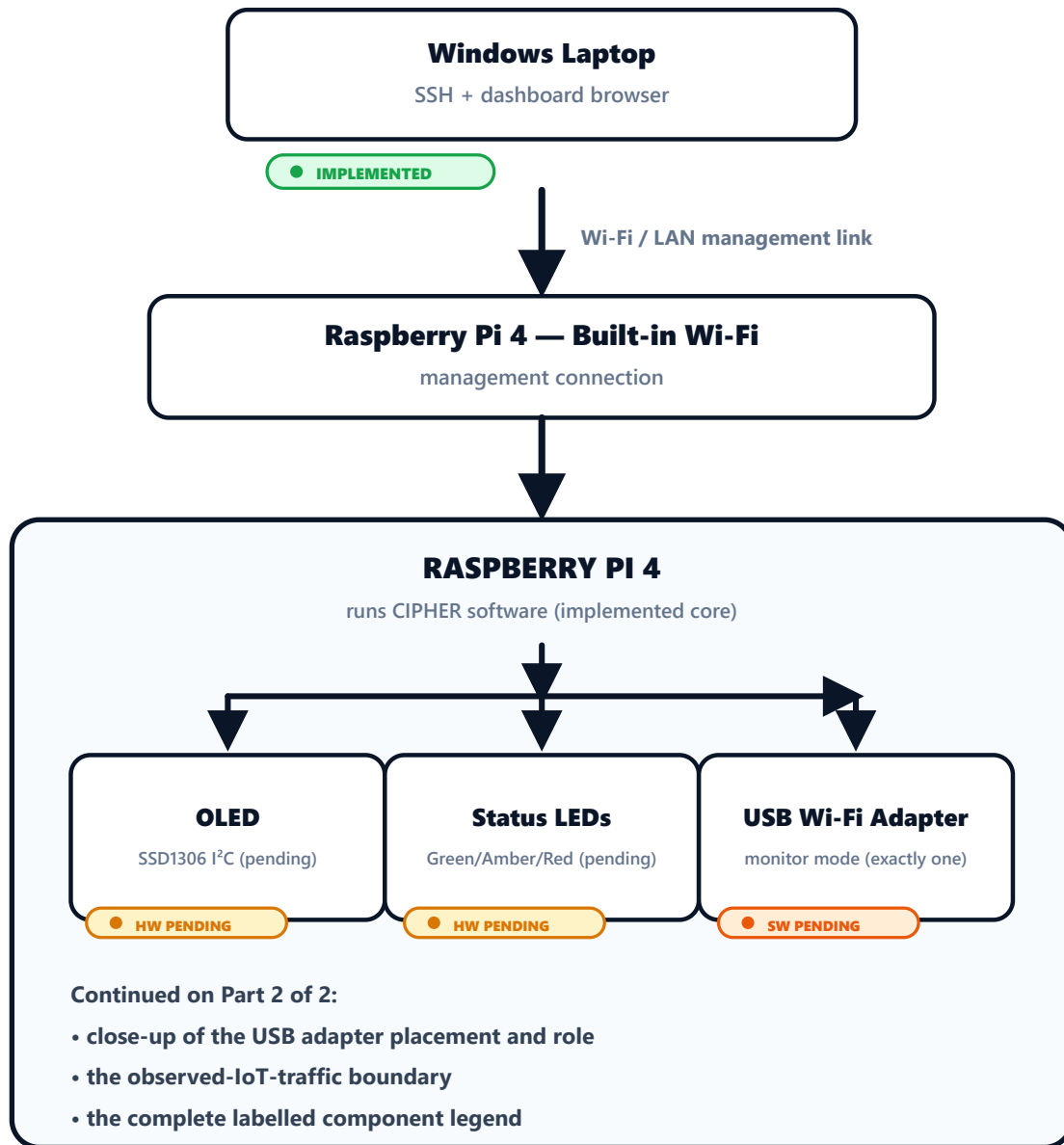
No .service unit file exists in the repository. The intended boot concept, at a conceptual level only:

- 1 Pi boots**
- 2 Network becomes ready**
- 3 CIPHER starts**
- 4 Health check passes**
- 5 Dashboard becomes available**

NO FINAL SERVICE FILE YET

This manual does not invent unit file contents, restart policies, or dependency ordering directives. Those are systemd implementation decisions that have not been made in the repository yet.

FINAL PHYSICAL ASSEMBLY — PART 1 OF 2



EXACTLY ONE external USB Wi-Fi adapter appears in this assembly.

Built-in Wi-Fi is always the management link — never used for capture.

FINAL PHYSICAL ASSEMBLY — PART 2 OF 2

Raspberry Pi 4

(continued)



- | | |
|--|---|
| 1 SSD1306 OLED
I ² C — status display
HARDWARE PENDING | 5 Resistors
one per LED, current-limiting
HARDWARE PENDING |
| 2 Green LED
LOW risk indicator (desired)
HARDWARE PENDING | 6 Jumper wires
male-to-female, GPIO header
HARDWARE PENDING |
| 3 Amber LED
MEDIUM risk indicator (desired)
HARDWARE PENDING | 7 External USB Wi-Fi Adapter
monitor-mode capture (exactly one)
SOFTWARE PENDING |
| 4 Red LED
HIGH risk indicator (desired)
HARDWARE PENDING | 8 Observed IoT Traffic
boundary — not physically wired
SOFTWARE PENDING |

Every numbered item above is cross-referenced in the final wiring table
(Section: Complete GPIO Assembly) once GPIO pins are approved.

Items 1–6 (OLED, LEDs, resistors, jumper wires):

HARDWARE INTEGRATION PENDING — GPIO pin map not yet approved (see dedicated page).

Do not wire these until the GPIO assignment page is finalized.

Before power

- ☐ [] microSD inserted
- ☐ [] GPIO wiring visually checked (if any is present — see pages 22–25)
- ☐ [] No 3.3V / 5V mix-up on any connection
- ☐ [] LED resistors installed (if LEDs are wired)
- ☐ [] LED polarity checked (if LEDs are wired)
- ☐ [] OLED wiring checked (if OLED is wired)
- ☐ [] Exactly ONE USB Wi-Fi adapter connected
- ☐ [] No loose conductive parts near the board

After power

- ☐ [] Pi boots
- ☐ [] Built-in Wi-Fi connects (management link)
- ☐ [] SSH works
- ☐ [] USB monitoring adapter detected
- ☐ [] CIPHER .venv activates
- ☐ [] Dependencies load
- ☐ [] `pytest -q` passes
- ☐ [] `preflight.py` passes
- ☐ [] `evaluate_demo.py` passes
- ☐ [] Dashboard reachable from the Windows laptop
- ☐ [] OLED hardware test passes — **• ONCE IMPLEMENTED**
- ☐ [] LED test passes — **• ONCE IMPLEMENTED**
- ☐ [] Capture test passes — **• ONCE IMPLEMENTED**

⚠ Pi does not boot

Check: power LED status, microSD fully seated

Likely cause: corrupt image, bad card, insufficient power

Safe next action: re-flash the microSD with Raspberry Pi Imager

⚠ Power warning shown

Check: the supply's rated wattage/amperage

Likely cause: under-rated or damaged USB-C supply/cable

Safe next action: use the official/rated Pi 4 power supply

⚠ SSH unavailable

Check: SSH was enabled during imaging; correct hostname/IP

Likely cause: SSH not enabled, or wrong address

Safe next action: re-check router client list; re-image if needed

⚠ Built-in Wi-Fi unavailable

Check: `ip link`, `iw dev` output

Likely cause: wrong SSID/credentials at imaging time

Safe next action: re-run Imager's Wi-Fi configuration step

⚠ USB Wi-Fi adapter not detected

Check: `lsusb` output, cable/port seating

Likely cause: unsupported chipset, faulty port

Safe next action: try another USB port; confirm adapter model

⚠ Monitor mode unsupported

Check: `iw list` supported interface modes

Likely cause: chipset/driver does not support monitor mode

Safe next action: record chipset; treat capture as pending

⚠ Driver missing for adapter

Check: `dmesg` for load errors after plugging in

Likely cause: out-of-tree driver not present in this OS image

Safe next action: confirm adapter's Linux driver support before proceeding

⚠ Dashboard inaccessible from laptop

Check: `FLASK_HOST`, Pi IP, same trusted network

Likely cause: still bound to 127.0.0.1

Safe next action: set `FLASK_HOST=0.0.0.0` and restart (page 21)

⚠️ OLED not detected

Check: I²C enabled, wiring against approved map (once frozen)

Likely cause: I²C disabled, or wiring not yet approved/frozen

Safe next action: confirm GPIO assignment page has been finalized first

⚠️ OLED blank

Check: power/GND connections, module orientation

Likely cause: loose connection, wrong I²C address

Safe next action: re-check each connection against the frozen wiring table

⚠️ LED not lighting / wrong colour

Check: polarity, resistor present, correct GPIO pin

Likely cause: reversed polarity or wrong pin

Safe next action: verify against the frozen wiring table, not memory

⚠️ Python dependency failure

Check: fresh `.venv`, exact `requirements.txt` pins

Likely cause: reused Windows `venv`, or partial install

Safe next action: recreate `.venv` per page 17

⚠️ ML model unavailable

Check: `ml/artifacts/anomaly_detector.joblib` present

Likely cause: model not trained on this machine yet

Safe next action: not a blocker — QRS-only mode is normal; retrain if needed (page 20)

⚠️ Capture permission error

Check: privileges required for raw packet capture on Linux

Likely cause: capture not implemented yet (page 26) or missing capability

Safe next action: treat as pending; do not grant broad root access ad hoc

⚠️ Report generation failure

Check: signing keys in `data/keys/`, disk space

Likely cause: incomplete ML-DSA-44 keypair

Safe next action: restore the missing key file or let CIPHER regenerate both

⚠️ Firewall/enforcement issue

Check: current backend in use

Likely cause: Linux enforcement is not implemented (by design, page 27)

Safe next action: confirm `NoOpIsolationBackend` is in use; do not apply manual firewall rules

Presentation / demonstration order

Follows existing CIPHER capabilities; the controlled demo remains the reliable fallback throughout.

1. Pi boots
2. SSH / management connection verified
3. CIPHER preflight (`preflight.py`)
4. Dashboard opened on the laptop
5. Controlled PCAP demo (`run_demo.py` fixture)
6. Live Pi capture — only if implemented and stable
7. OLED / LEDs — only if implemented
8. Generated, signed PDF report
9. Enforcement decision demo, per the current safe implementation

CAPABILITY	CURRENT STATUS	VERIFIED ON PI?
Core CIPHER pipeline (capture → fingerprint → entropy → risk)	• IMPLEMENTED	Yes — at the 649-test baseline
Quantum Risk Score (QRS)	• IMPLEMENTED	Yes — at the 649-test baseline
Isolation Forest (optional)	• IMPLEMENTED	Yes — artifact trained on the Pi; preflight passed
Signed PDF report generation	• IMPLEMENTED	Covered by the Pi test run
ML-DSA-44 signing	• IMPLEMENTED	Covered by the Pi test run
REST API + dashboard	• IMPLEMENTED	Not yet confirmed from the laptop browser
Built-in management Wi-Fi	• OS-LEVEL, IMPLEMENTED	Yes — SSH and management worked throughout
Current 1192-test baseline	• IMPLEMENTED (WINDOWS)	No — not yet pulled or re-verified on the Pi
USB monitor adapter — hardware	• HARDWARE CAPABILITY VERIFIED	Yes — monitor mode + tcpdump PCAP
USB monitor adapter — CIPHER ingestion	• SOFTWARE PENDING	No — no Radiotap/802.11 capture source exists
Multi-device live capture	• SOFTWARE PENDING	No
Pi performance benchmark	• HARNESS IMPLEMENTED	No — pending execution
OLED	• HARDWARE INTEGRATION PENDING	No — no driver code
Status LEDs	• HARDWARE INTEGRATION PENDING	No — no driver code
Linux enforcement (firewall isolation)	• SOFTWARE PENDING	No — NoOpIsolationBackend only
Physical isolation latency	• NOT MEASURED	No
systemd service	• SOFTWARE PENDING	No
Sustained (72-hour) operation	• NOT YET VALIDATED	No

POPULATE TRUTHFULLY

Fill the "Verified on Pi?" column only from actual repository inspection and actual tests run on real Pi hardware — never from assumption or from Windows results.